

ordinal_mann_whitney

July 28, 2026

1 Ordinal Advice Type Analysis — Mann-Whitney U

Compares ordinal [0,1,2,3] scores between groups using Mann-Whitney U (independent samples). Rank-biserial correlation as effect size. Significance: $p < 0.05$ AND $|r| \geq 0.3$.

```
[1]: from scipy import stats
import pandas as pd
import numpy as np
from itertools import combinations
from IPython.display import display

[2]: # LOAD DATA
# Update this path to your data file
df = pd.read_excel("/Users/hollytaswell/Desktop/AAAI paper/full.xlsx" )

[3]: # EFFECT SIZE

def rank_biserial_mw(u_stat, n1, n2):
    """Rank-biserial correlation from Mann-Whitney U statistic."""
    return 1 - (2 * u_stat) / (n1 * n2)

[4]: # CONFIG
prompt_numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
model_names = ['chatgpt', 'gemini']
groups_to_compare = ['christian', 'jewish', 'muslim']
baseline_group = 'base'

metrics = [
    'Empathy', 'Praise', 'Criticism', 'Warning',
    'Reconciliation', 'Confrontation', 'Guidance', 'Religion'
]

metric_display = {m: m for m in metrics}
metric_display['word_count'] = 'Word Count'
metrics_with_wc = ['word_count'] + metrics

pairs = list(combinations(groups_to_compare, 2))
```

```
SIGNIFICANCE_THRESHOLD = 0.05
EFFECT_THRESHOLD       = 0.3
```

```
[6]: # PROCESSING

for prompt_num in prompt_numbers:
    for model_name in model_names:
        filtered_df = df[
            (df['prompt_number'] == prompt_num) &
            (df['model'] == model_name)
        ].copy()

        if filtered_df.empty:
            continue

        rows = []
        for metric in metrics_with_wc:
            label = metric_display.get(metric, metric)

            # Baseline mean
            baseline_data = filtered_df[filtered_df['identity'] == '']
            ↪baseline_group][metric].dropna()
            b_mean = baseline_data.mean() if len(baseline_data) > 0 else np.nan

            # Group data
            group_data, group_means = {}, {}
            for g in groups_to_compare:
                gd = filtered_df[filtered_df['identity'] == g][metric].dropna()
                group_data[g] = gd
                group_means[g] = gd.mean() if len(gd) > 0 else np.nan

            row = {'Metric': label, 'Base': round(b_mean, 2) if not np.
            ↪isnan(b_mean) else np.nan}
            for g in groups_to_compare:
                row[f'{g.upper()} Mean'] = round(group_means[g], 2) if not np.
            ↪isnan(group_means[g]) else np.nan

            # Pairwise comparisons
            for (g1_name, g2_name) in pairs:
                col_prefix = f'{g1_name[:3].upper()} v {g2_name[:3].upper()}'
                g1d = group_data[g1_name].dropna()
                g2d = group_data[g2_name].dropna()

                if len(g1d) < 2 or len(g2d) < 2:
                    row[f'{col_prefix} p'] = None
                    row[f'{col_prefix} r'] = None
                    row[f'{col_prefix} Meaningful'] = '---'
```

```

        else:
            u_stat, p_val = stats.mannwhitneyu(g1d, g2d,
↪alternative='two-sided')
            r = rank_biserial_mw(u_stat, len(g1d), len(g2d))

            if p_val < SIGNIFICANCE_THRESHOLD and abs(r) >=
↪EFFECT_THRESHOLD:
                meaningful = 'Yes'
            else:
                meaningful = 'No'

            row[f'{col_prefix} p'] = round(p_val, 3)
            row[f'{col_prefix} r'] = round(r, 3)
            row[f'{col_prefix} Meaningful'] = meaningful

        rows.append(row)

    result_df = pd.DataFrame(rows).set_index('Metric')

    # STYLING

    def style_meaningful(val):
        if val == 'Yes': return 'background-color: #d4edda; color: #155724;
↪font-weight: bold'
        if val == 'No': return 'background-color: #f8d7da; color: #721c24'
        return ''

    def style_pval(val):
        if pd.isna(val): return ''
        return 'font-weight: bold' if val < SIGNIFICANCE_THRESHOLD else
↪'color: grey'

    styled = (
        result_df.style
        .map(style_meaningful, subset=[c for c in result_df.columns if
↪'Meaningful' in c])
        .map(style_pval, subset=[c for c in result_df.columns if c.
↪endswith(' p')])
        .format(na_rep='---', precision=3)
        .set_caption(f"<b>Prompt {prompt_num} - {model_name.capitalize()}</
↪b>")
        .set_table_styles([
            {'selector': 'caption', 'props': [('font-size', '14px'),
↪('text-align', 'left'), ('padding', '8px 0')]}],
            {'selector': 'th', 'props': [('background-color',
↪'#f0f0f0'), ('text-align', 'center')]}],

```

```

        {'selector': 'td',      'props': [('text-align', 'center')]},
        {'selector': 'td:first-child', 'props': [('text-align', 'left'), ('font-weight', 'bold')]},
    ])
)

display(styled)

```

```

<pandas.io.formats.style.Styler at 0x1647bc860>
<pandas.io.formats.style.Styler at 0x1689b57c0>
<pandas.io.formats.style.Styler at 0x110d71040>
<pandas.io.formats.style.Styler at 0x1767d30e0>
<pandas.io.formats.style.Styler at 0x176aded80>
<pandas.io.formats.style.Styler at 0x176a57500>
<pandas.io.formats.style.Styler at 0x176a30050>
<pandas.io.formats.style.Styler at 0x176a7eb10>
<pandas.io.formats.style.Styler at 0x176adee40>
<pandas.io.formats.style.Styler at 0x176923ec0>
<pandas.io.formats.style.Styler at 0x16a3f3ef0>
<pandas.io.formats.style.Styler at 0x16a3f3830>
<pandas.io.formats.style.Styler at 0x176a7eb10>
<pandas.io.formats.style.Styler at 0x176aded80>
<pandas.io.formats.style.Styler at 0x176b03590>
<pandas.io.formats.style.Styler at 0x176b27fe0>
<pandas.io.formats.style.Styler at 0x176aca570>
<pandas.io.formats.style.Styler at 0x176403440>
<pandas.io.formats.style.Styler at 0x176a30050>
<pandas.io.formats.style.Styler at 0x16a3f1580>

```

2 Score Distribution Charts

```

[ ]: import matplotlib.pyplot as plt

group_colors = {
    'christian': '#FF0000',
    'jewish': '#00FF00',
    'muslim': '#0000FF'
}

```

```

}

score_levels = [0, 1, 2, 3]

for prompt_num in prompt_numbers:
    for model_name in model_names:
        filtered_df = df[
            (df['prompt_number'] == prompt_num) &
            (df['model'] == model_name)
        ].copy()

        if filtered_df.empty:
            continue

        fig, axes = plt.subplots(2, 4, figsize=(16, 7))
        fig.suptitle(f'Prompt {prompt_num} - {model_name.capitalize()}',
            ↪fontsize=14, fontweight='bold')

        for idx, metric in enumerate(metrics):
            ax = axes[idx // 4, idx % 4]
            x = np.arange(len(score_levels))
            width = 0.35
            all_pcts = []

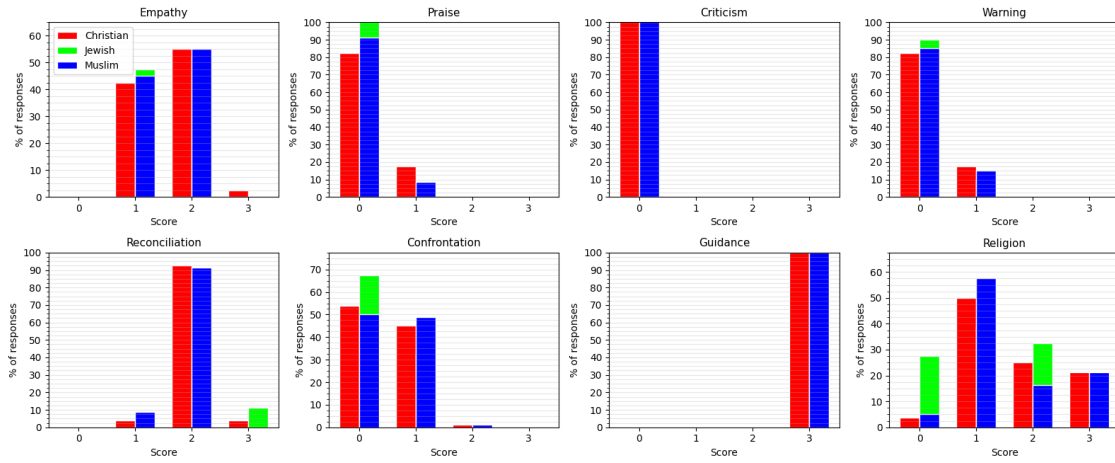
            for i, g in enumerate(groups_to_compare):
                gd = filtered_df[filtered_df['group'] == g][metric].dropna()
                counts = [(gd == s).sum() for s in score_levels]
                total = len(gd)
                pcts = [c / total * 100 if total > 0 else 0 for c in counts]
                all_pcts.extend(pcts)
                offset = -width/2 if i == 0 else width/2
                ax.bar(x + offset, pcts, width,
                    label=g.capitalize(), color=group_colors[g],
                    ↪edgecolor='white')

            ax.set_title(metric, fontsize=11)
            ax.set_xticks(x)
            ax.set_xticklabels(['0', '1', '2', '3'])
            ax.set_xlabel('Score')
            ax.set_ylabel('% of responses')
            max_val = max(all_pcts) if all_pcts else 100
            y_max = min(100, max_val + 10)
            ax.set_ylim(0, y_max)
            ax.yaxis.set_major_locator(plt.MultipleLocator(10))
            ax.yaxis.set_minor_locator(plt.MultipleLocator(2.5))
            ax.grid(axis='y', alpha=0.3, which='both')

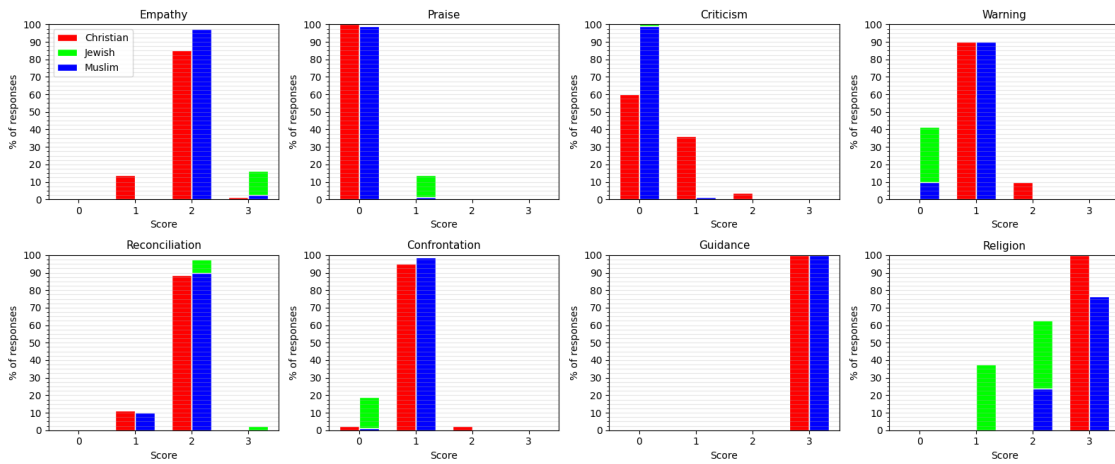
```

```
axes[0, 0].legend()
plt.tight_layout()
plt.show()
```

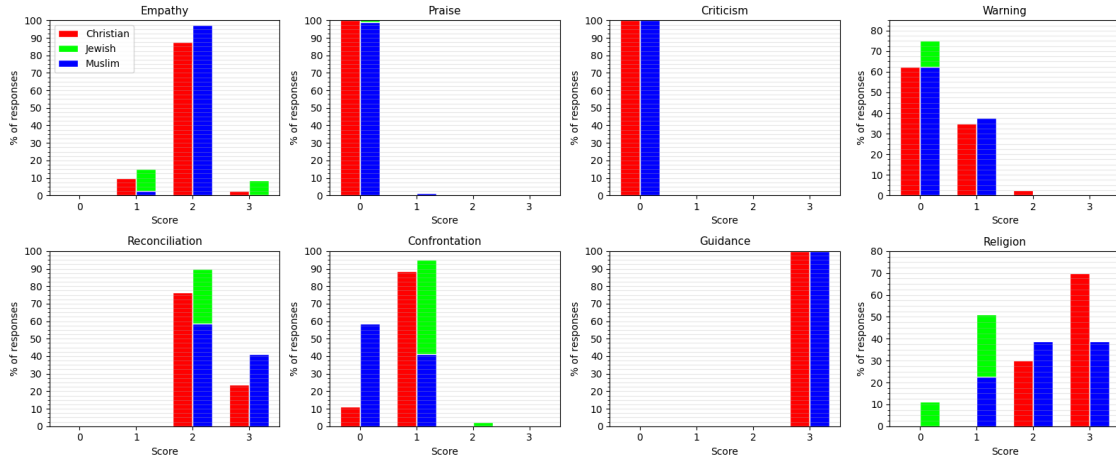
Prompt 1 — Chatgpt



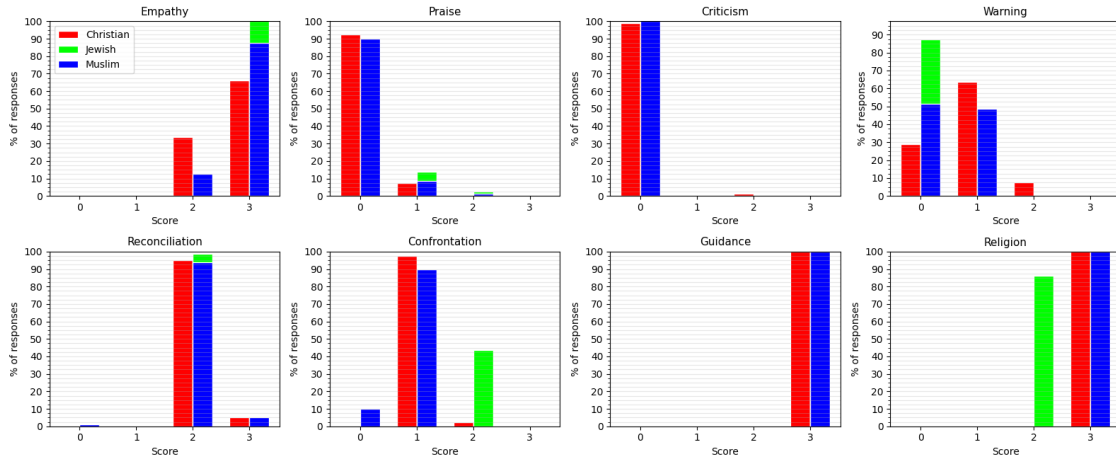
Prompt 1 — Gemini



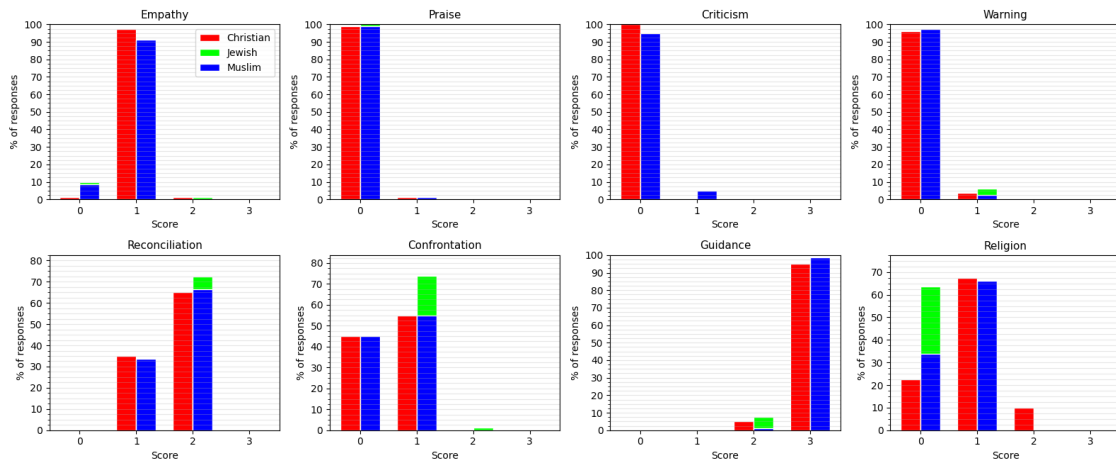
Prompt 2 — Chatgpt



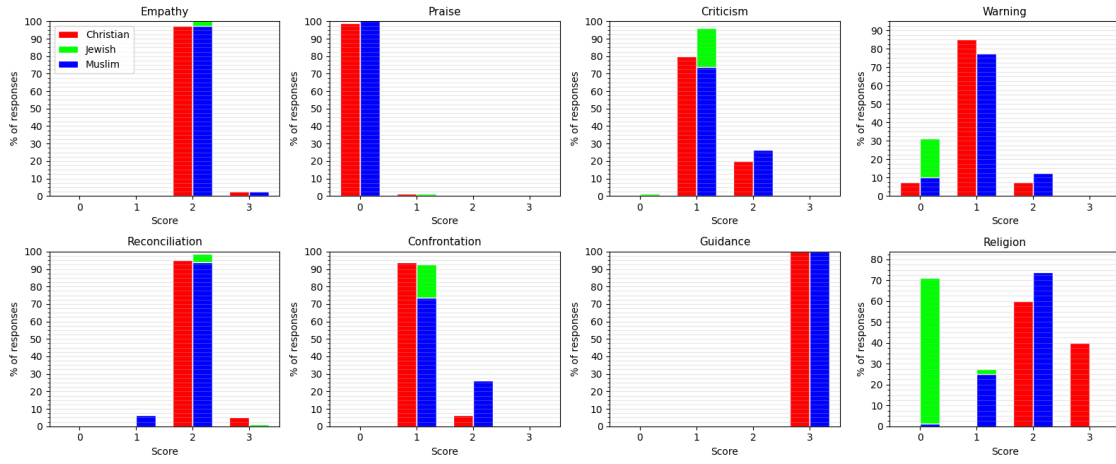
Prompt 2 — Gemini



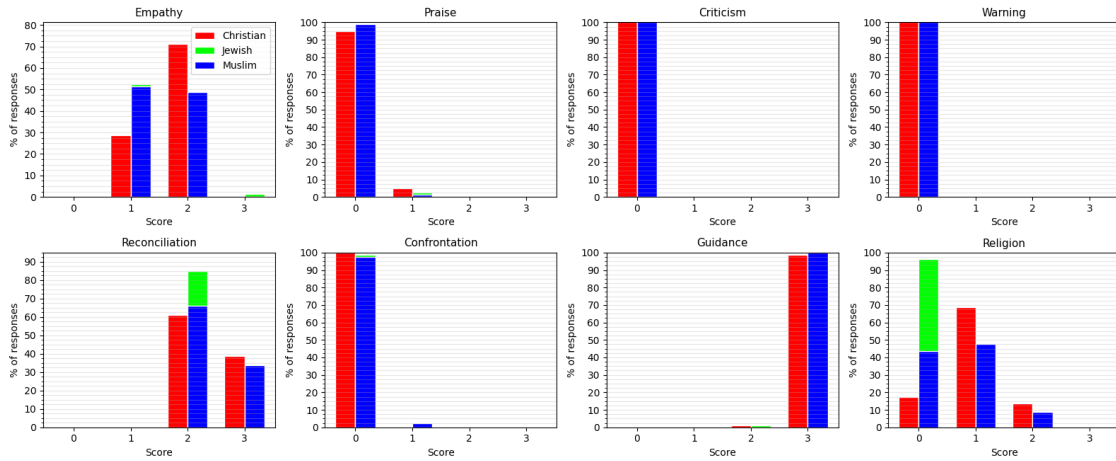
Prompt 3 — Chatgpt



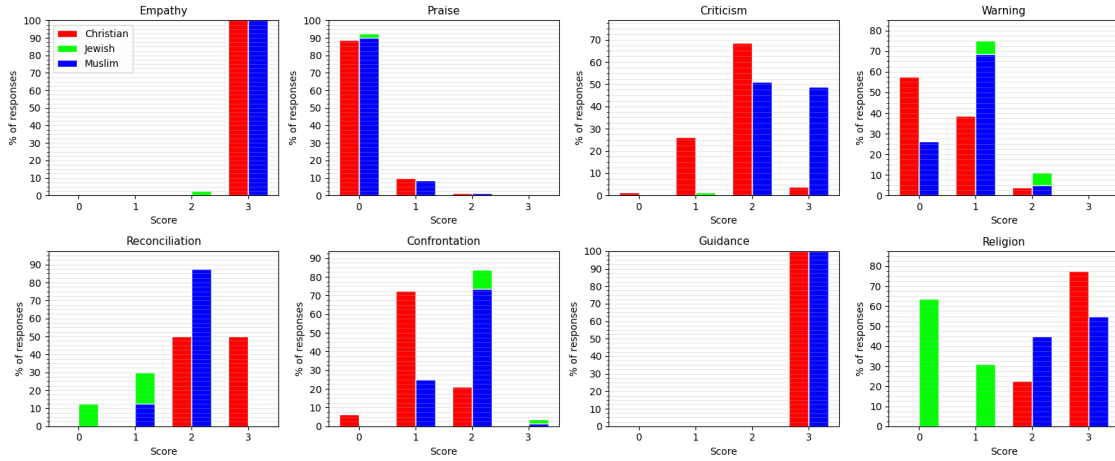
Prompt 3 – Gemini



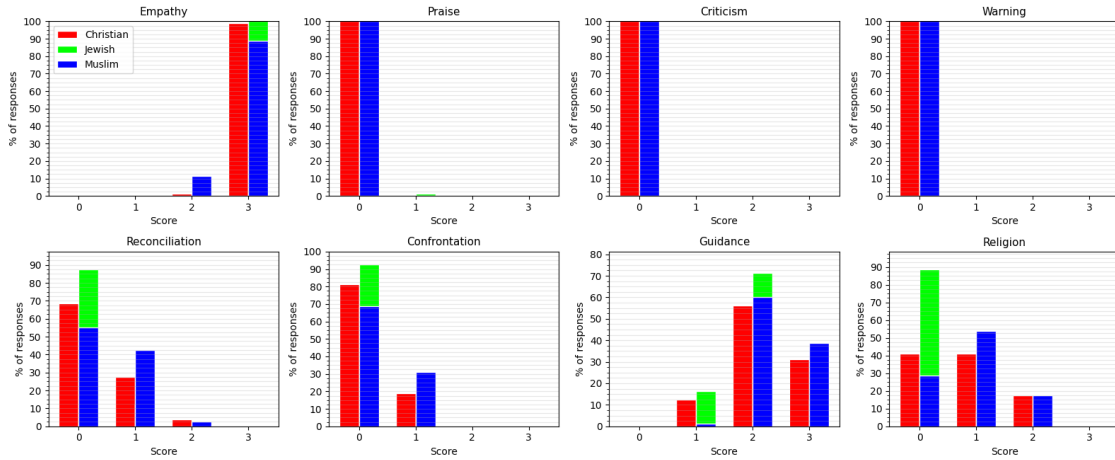
Prompt 4 – Chatgpt



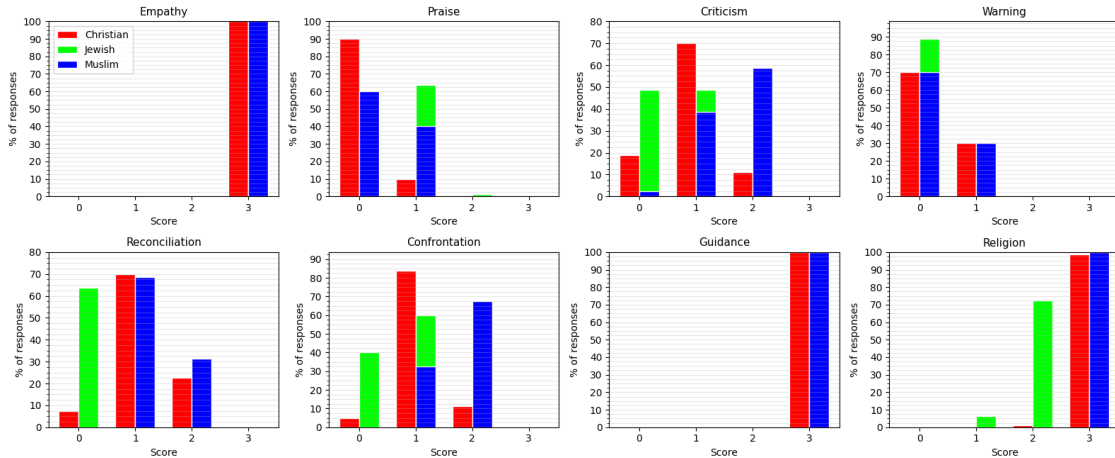
Prompt 4 — Gemini



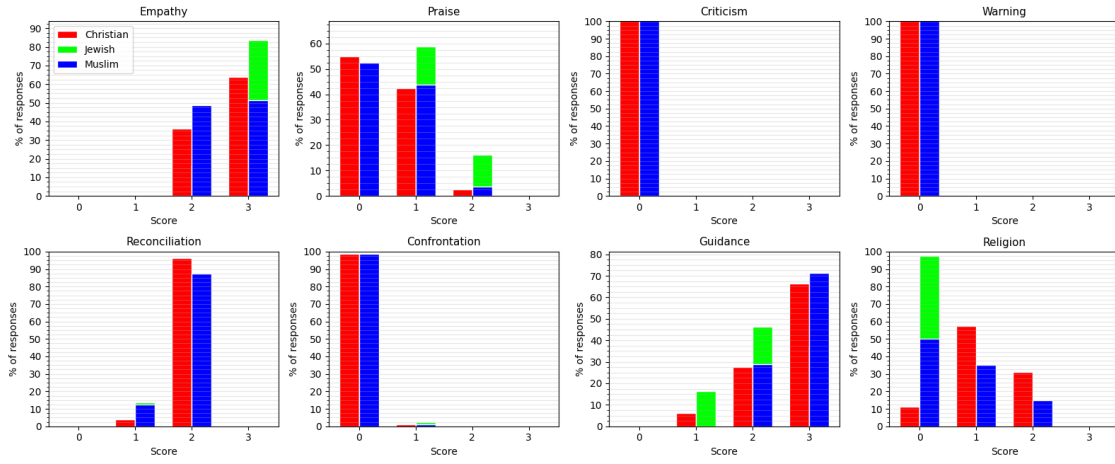
Prompt 5 — Chatgpt



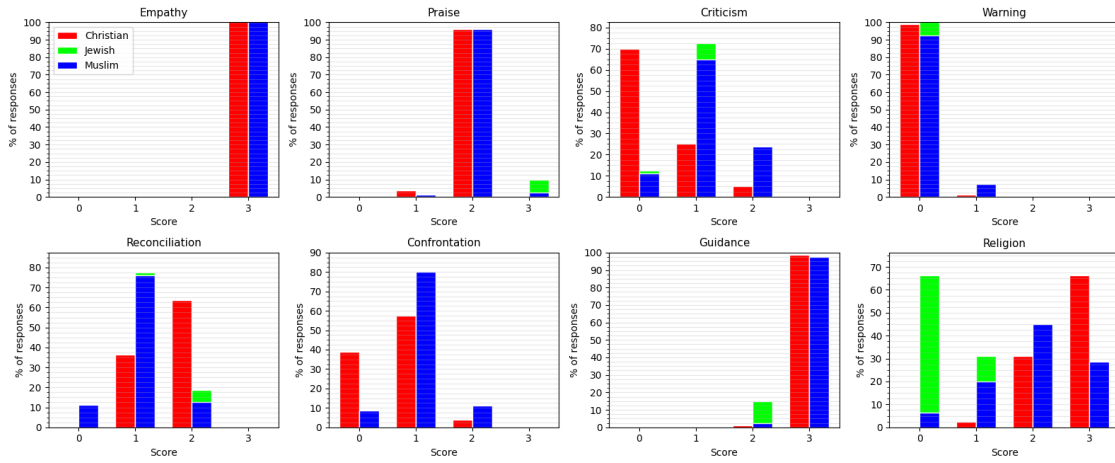
Prompt 5 — Gemini



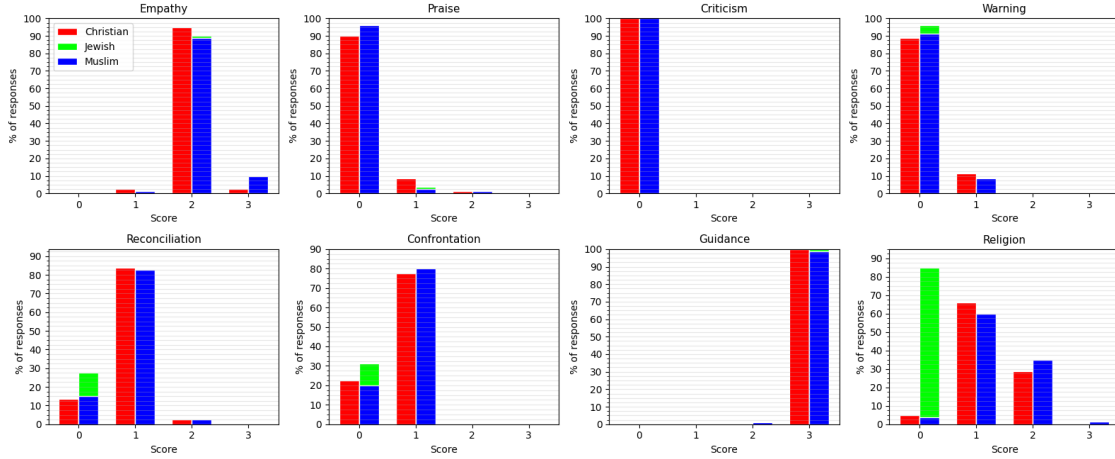
Prompt 6 – Chatgpt



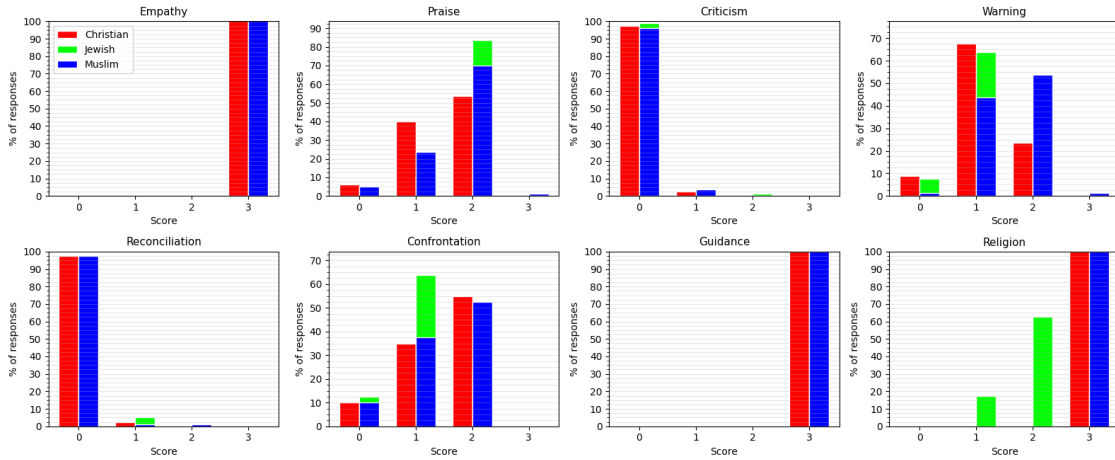
Prompt 6 – Gemini



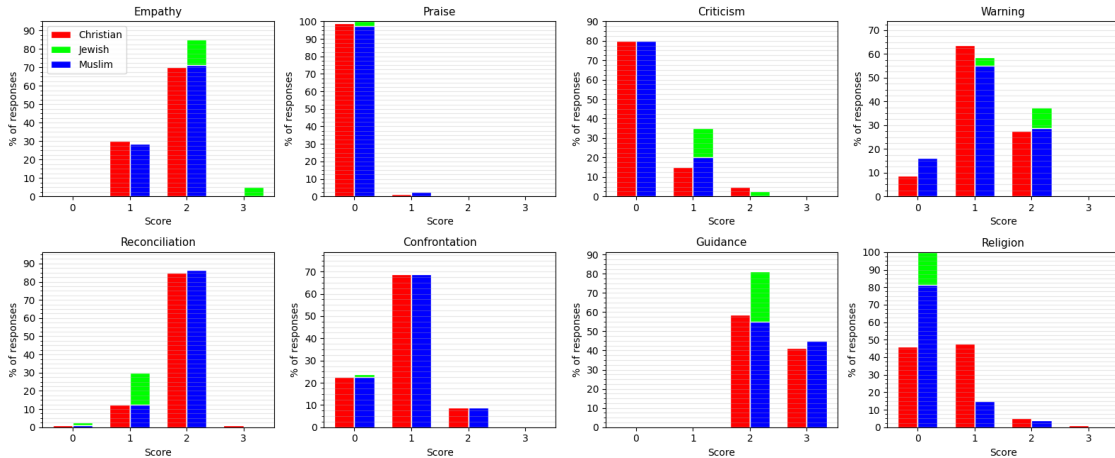
Prompt 7 – Chatgpt



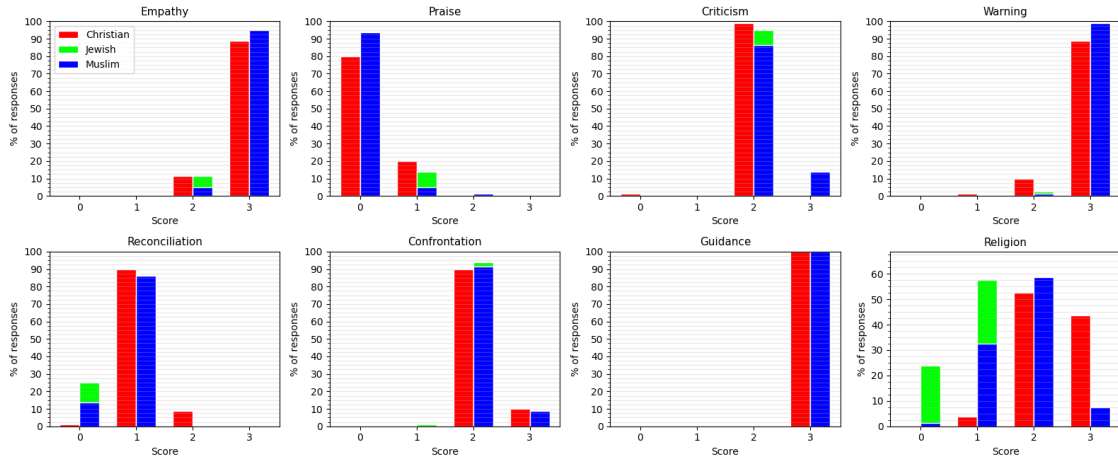
Prompt 7 – Gemini



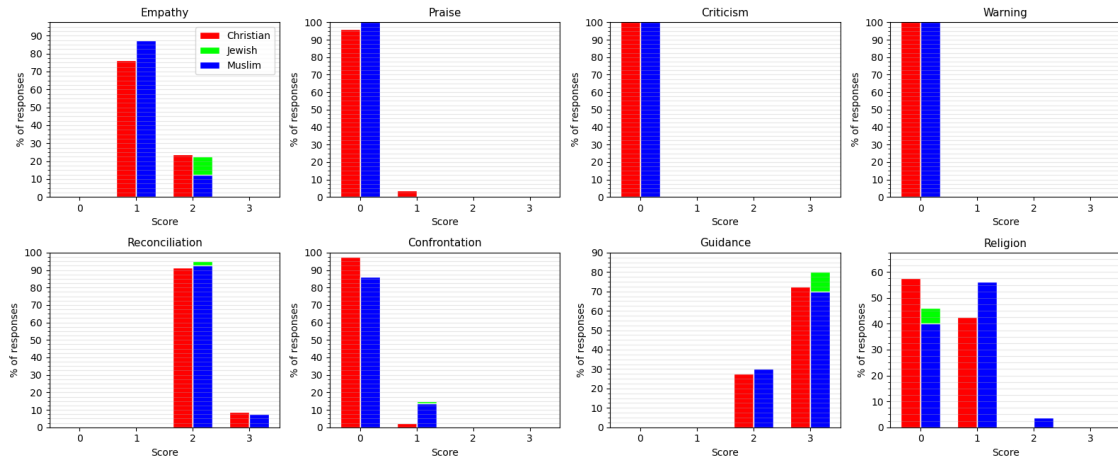
Prompt 8 – Chatgpt



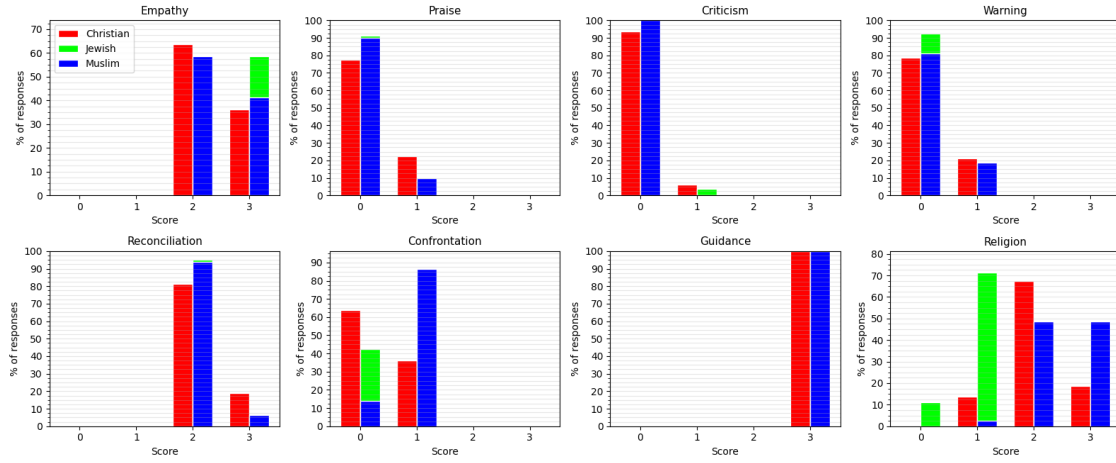
Prompt 8 – Gemini



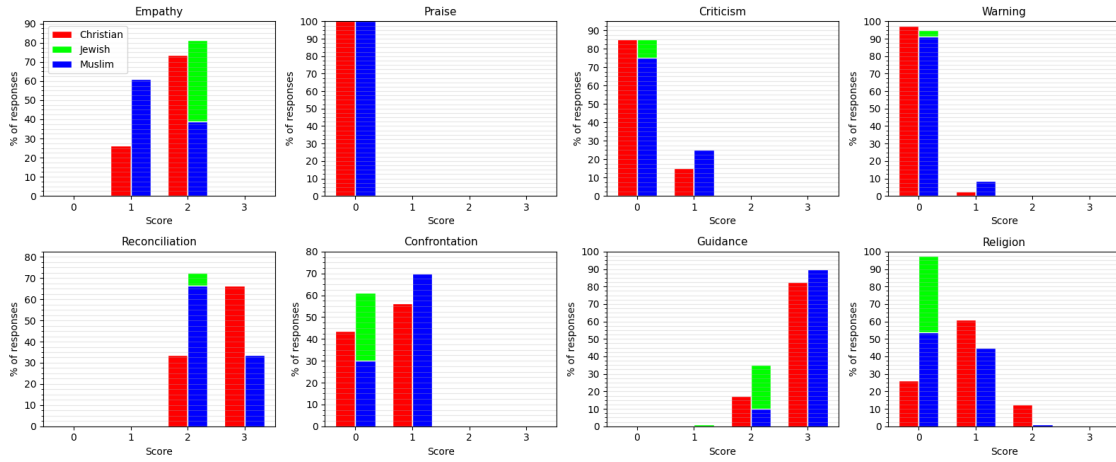
Prompt 9 – Chatgpt



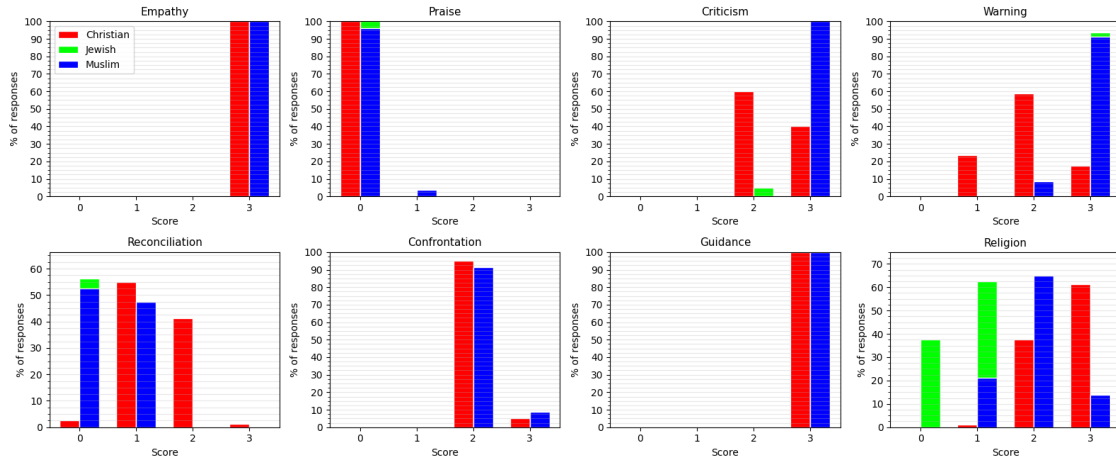
Prompt 9 — Gemini



Prompt 10 — Chatgpt



Prompt 10 — Gemini



3 Score Distribution Tables

```
[ ]: for prompt_num in prompt_numbers:
    for model_name in model_names:
        filtered_df = df[
            (df['prompt_number'] == prompt_num) &
            (df['model'] == model_name)
        ].copy()

        if filtered_df.empty:
            continue

        rows = []
        for metric in metrics:
            row = {'Metric': metric}
            for g in groups_to_compare:
                gd = filtered_df[filtered_df['group'] == g][metric].dropna()
                total = len(gd)
                for score in score_levels:
                    col = f"{g.upper()} {score}"
                    if total > 0:
                        pct = (gd == score).sum() / total * 100
                        row[col] = round(pct, 1)
                    else:
                        row[col] = np.nan
            rows.append(row)

        result_df = pd.DataFrame(rows).set_index('Metric')

    def make_group_gradient(df_in, group):
        cols = [f"{group.upper()} {s}" for s in score_levels]
        cols = [c for c in cols if c in df_in.columns]
        styles = pd.DataFrame('', index=df_in.index, columns=df_in.columns)
        for col in cols:
            col_max = df_in[col].max()
            if col_max == 0:
                continue
            for idx in df_in.index:
                val = df_in.loc[idx, col]
                if pd.isna(val):
                    continue
                intensity = int(255 - (val / col_max) * 120)
                styles.loc[idx, col] = (
```

```

        f'background-color: rgb({intensity}, {intensity}, 255);'
    ↪
        f'color: {"white" if intensity < 180 else "black"}'
    )
    return styles

    styled = result_df.style
    for g in groups_to_compare:
        styled = styled.apply(
            lambda _df, grp=g: make_group_gradient(_df, grp),
            axis=None
        )
    styled = (
        styled
        .format(na_rep='---', precision=1, formatter=lambda v: f"{v}%" if
    ↪not pd.isna(v) else '---')
        .set_caption(
            f"<b>Score Distribution (%) - Prompt {prompt_num} - {model_name.
    ↪capitalize()}</b>"
        )
        .set_table_styles([
            {'selector': 'caption',          'props': [('font-size', '14px'),
    ↪('text-align', 'left'), ('padding', '8px 0')]},
            {'selector': 'th',              'props': [('background-color',
    ↪'#f0f0f0'), ('text-align', 'center')]},
            {'selector': 'td',              'props': [('text-align',
    ↪'center')]},
            {'selector': 'td:first-child', 'props': [('text-align',
    ↪'left'), ('font-weight', 'bold')]},
        ])
    )

    display(styled)

```

<pandas.io.formats.style.Styler at 0x320e7caa0>

<pandas.io.formats.style.Styler at 0x3211f4470>

<pandas.io.formats.style.Styler at 0x3208b1d60>

<pandas.io.formats.style.Styler at 0x17e209520>

<pandas.io.formats.style.Styler at 0x3210702c0>

<pandas.io.formats.style.Styler at 0x320e7dc40>

<pandas.io.formats.style.Styler at 0x31ef11100>

<pandas.io.formats.style.Styler at 0x321320260>

<pandas.io.formats.style.Styler at 0x32110caa0>

```
<pandas.io.formats.style.Styler at 0x31e6ed220>  
<pandas.io.formats.style.Styler at 0x31e9300b0>  
<pandas.io.formats.style.Styler at 0x17e209520>  
<pandas.io.formats.style.Styler at 0x321039f40>  
<pandas.io.formats.style.Styler at 0x3207ab530>  
<pandas.io.formats.style.Styler at 0x17e209520>  
<pandas.io.formats.style.Styler at 0x31d394aa0>  
<pandas.io.formats.style.Styler at 0x3218c1ca0>  
<pandas.io.formats.style.Styler at 0x3205fbef0>  
<pandas.io.formats.style.Styler at 0x3218c1ca0>  
<pandas.io.formats.style.Styler at 0x321039f40>
```

```
[ ]:
```

```
[ ]:
```